Inducție Structurală

11 decembrie 2024

Tipuri de date abstracte (TDA)

Un TDA specifică:

- un set de **date (valori care aparțin tipului respectiv)** (ex: O valoare a TDA-ului Stack pentru numere întregi este o stivă care conține elementele 2 și 3, iar toate valorile pentru TDA-ul Stack sunt toate stivele posibile.)
- un set de **operații** care se pot face cu acele date (ex: Pentru TDA-ul Stack operațiile posibile sunt pop și push, prin intermediul cărora generăm noi valori ale TDA-ului.)

De ce "abstract"?

- **independent de implementare** (același TDA poate fi implementat în diverse moduri care respectă definiția)
- vizibil ca model matematic sau programat ca interfață
- abstractizarea structurilor de date prin TDA-uri este importantă pentru că permite programatorilor să folosească structuri de date fără să știe implementarea lor (ex: pop(push(S, x)) va avea același comportament indiferent de limbajul/biblioteca folosită care implementează TDA-ul Stack. Astfel programatorii pot folosi TDA-uri cunoscând doar anumite proprietăți ale operatorilor unui TDA)

Disclaimer: in exemplul de mai jos nu a fost demonstrat (in acest suport de seminar) că

(∗) Toate datele tipului sunt produse de constructorii de bază intr-un mod unic (că există o funcție bijectivă între toate datele tipului si reprezentări simbolice care sunt formate doar din constructori de bază)

deși acest lucru este necesar pentru ca o inductie structurală care se refera doar la constructorii de baza sa funcționeze corect si sa duca la rezultate corecte în general.

Comentariu:
Pentru a demonstra ce a fost spus mai sus trebuie considerată relația de echivalență a mai multor reprezentări simbolice, care reyultă din axiome. Un alt mod de a adresa aceste lucruri este folosind așa-numite specificări formale pentru tipurile de date abstracte si așa-numite algebre pentru tipurile de date (care sunt instanțe ale specificarii formale/tipului de date abstract), in care context se formează un așa-numit quotient care descrie elementele specificării formale prin clase de echivalență in urma aplicării relațiilor de echivalenta din axiome (adică in quotient exista un singur element pentru mai multe elemente echivalente in contextul axiomelor, si se specifica un reprezentant pentru fiecare clasa de echivalența). In urma acestei operatii demonstrațiile despre un tip de date abstract cum ar fi Nat se pot reduce la demonstrații despre reprezentanții claselor de echivalență din quotient, in cazul Nat acesti reprezentanți ar fi zero, succ(zero), si toate reprezentarile simbolice formate doar din succ si zero. Quotientul poate fi scris $\{[zero], [succ(zero)], [succ(succ(zero))], \dots\}$, continand deci clasele de echivalenta ale acestor reprezentari simbolice conform axiomelor intr-o specificare formala care descrie numere naturale. Parantezele patrate ([ ]) in jurul unei reprezentări simbolice denotă clasa ei de echivalență conform axiomelor.

Dacă nu se demonstrează (∗), sau dacă nu se știe că acest lucru este valabil pentru că a demonstrat cineva anterior acest lucru, o inducție structurală corectă va trebui facută pentru toate reprezentările simbolice ale unui tip de date abstract, ca inducăie

completă peste numarul de operatori (inclusiv constructori de bază) din reprezentarea simbolică, in cazul Nat de mai jos adresând și reprezentări simbolice ale tipului formate cu operatorul add, complicând semnificativ inducția strcuturală pentru că sunt adresate mai multe moduri de a construi elemente de tip Nat in cadrul demonstrației (față de cazul in care sunt adresate doar reprezentări simbolice formate din doar succ si zero).

***O inducție completă este corectă in baza teoremei inducției complete, deci acesta este un mod valabil de a demonstra lucruri despre tipul de date.* In acest caz complexitatea demonstrației ca există functia bijectivă intre reprezentări simbolice formate din constructori de baza și elementele tipului de date (sau, respectiv, și reprezentații claselor de echivalență din quotient), este transferată inspre inducție.**

Avantajul de a demonstra că tipul de date abstract (sau specificarea formală) conține reprezentanți ai fiecărei clase de echivalență rezultate din axiome (prin formarea quotientului) formati dintr-o submulțime a operatorilor(inclusiv constructori) care pot genera elemente ale tipului duce la faptul ca inducția structurală sau completă pentru fiecare tip de date specificat prin tipul de date abstract poate adresa doar reprezentări simbolice care produc tipul de date folosind elemente ale submulțimii respective, ignorând ceilalți operatori care pot genera elemente ale tipului de date (de exemplu in cazul Nat, operatorul add)

Nu este indeajuns ca reprezentările simbolice formate din constructori de bază să reprezinte toate elementele tipului de date (deci sa avem o funcție doar surjectivă de la reprezentări simbolice la elemente), pentru că atunci s-ar putea crea un tip de date Nat ca mai jos, cu add ca si constructor de baza, caz in care "dimensiunea" unui element al tipului de date este incorect definită ca fiind numărul de constructori de bază din reprezentarea simbolică, incorect pentru ca succ(succ(zero)) este egal cu add(succ(zero), succ(zero)), adică pentru același element al tipului de date exista 2 versiuni diferite ale dimensiunii, si anume 2 si 3. Dacă se demonstrează o functie surjectivă, se poate omite definiția dimensiunii, si se poate face inducție completă peste numărul de operatori(inclusiv constructori) din reprezentările simbolice, ducând la o demonstratie corectă, fara complicații cu un cuvant in plus numit dimensiune. Observăm totuși că in cazul formării quotientului pentru specificarea formală, demonstrația ca există o funcție bijectivă intre reprezentanții claselor de echivalență si elementele tipului de date are loc, dacă in quotient este trecut un singur reprezentant al fiecărei clase de echivalență.

Exemplu – Tipul de date abstract Nat

Nume: Nat
Descriere: numere naturale
Tipuri importate: Bool

Constructori de bază:

- $zero :\rightarrow Nat$ # orice număr natural este fie 0
- $succ : Nat \rightarrow Nat$ # fie succesorul altui număr natural

---

Operatori:

- $zero? : Nat \rightarrow Bool$ # testează dacă un număr natural este 0
- $add : Nat * Nat \rightarrow Nat$ # adună 2 numere naturale

---

Axiome:

- $zero?(zero) = true$
- $zero?(succ(n)) = false$
- $add(zero, n) = n$
- $add(succ(m), n) = succ(add(m, n))$

Noțiuni teoretice TDA-uri

Constructori de tip

- toți operatorii unui tip t care **produc o valoare a lui t** se numesc constructori ai acelui tip
- **zero, succ, add** sunt, toți, constructori ai tipului Nat
- zero? nu este constructor pentru Nat, întrucât produce un Bool

Constructori de bază

- un set de constructori **necesari și suficienți** pentru a obține **toate valorile tipului**
- **zero** și **succ** sunt constructori de bază pentru Nat
- add nu este constructor de bază, întrucât nu e necesar; orice număr natural se poate obține din aplicări succesive de zero sau succ

Constructorii unui tip t, din punct de vedere al **parametrilor**

- **nulari**: au 0 argumente; fie Σ_0 mulțimea constructorilor de bază nulari (ex: zero pentru tipul Nat)
- **externi**: au un număr de argumente, dintre care **niciunul nu este de tip t**; fie Σ_e mulțimea constructorilor de bază externi (nu există constructor extern în fișa tipului Nat)
- **interni**: au un număr de argumente, dintre care **cel puțin unul este de tip t**; fie Σ_i mulțimea constructorilor de bază interni (ex: **succ** pentru tipul Nat)

Axiome

- rolul lor este să specifice **cum se comportă toți operatorii TDA-ului pe toate valorile TDA-ului**
- pentru aceasta, e suficient să specifice cum se comportă pe toți **constructorii de bază** (întrucât orice valoare a TDA-ului se poate obține numai din constructorii de bază)
- este extrem de important ca **axiomele să "acopere" toate valorile tipului**
- de asemenea ele trebuie să fie scrise **neredundant** (de exemplu, la axiomele pentru add, am acoperit toate posibilitățile prin a varia doar primul parametru între zero și succ(m), nu era nevoie să procedăm la fel și pentru al doilea; în general, e foarte important (și ușurează mult demonstrațiile) să găsim exprimarea minimală și completă pentru axiome; în particular, când există mai mulți parametri de tip t, considerăm că scriem axiomele din punctul de vedere al unuia dintre ei, nu din al tuturor)

Dimensiunea unei valori v a unui TDA t

- reprezintă numărul aparițiilor constructorilor lui t în formula care desemnează pe v
- ex: succ(succ(zero)) are dimensiunea 3, add(succ(zero), succ(zero)) are dimensiune 4

Inducție structurală

Când este adecvată?

- când algoritmul (sau o parte din el) primește date de un tip t care satisfac o proprietate P (conform aserțiunii de intrare) și trebuie să producă date de tip t care satisfac, de asemenea, proprietatea P (conform aserțiunii de ieșire)
- faptul că proprietatea P este păstrată în urma aplicării algoritmului poate fi demonstrat prin inducție structurală

Moduri de înțelegere a inducției structurale

1. pentru a dovedi că proprietatea P va fi respectată în urma aplicării algoritmului pe toate valorile posibile de tip t, va trebui să arătăm că e respectată de **toți constructorii de bază**

2. pentru a dovedi ca proprietatea P va fi respectată în urma aplicării algoritmului pe toate valorile posibile de tip t, vom face o **inducție după dimensiunea valorilor tipului t** (în cazul de bază vor fi constructorii **nulari** și **externi**, întrucât aceștia sunt singurii de dimensiune 1; **pasul de inducție** se va referi la **constructorii interni**, întrucât aceștia au proprietatea de a incrementa dimensiunea (P(n)→P(n+1), ca la inducția matematica))

Ambele viziuni sunt surprinse de aceeași schemă de inducție structurală:

Cazuri de baza:

$\forall \sigma \in \Sigma_0 \bullet P(\sigma)$

$\forall \sigma \in \Sigma_e, d \in Dom_\sigma \bullet P(\sigma(d))$

Pas de inducție:

$\forall x \in t \bullet P(x) \rightarrow P(\sigma(..., x, ...)), \forall \sigma \in \Sigma_i$

$atunci\ \forall x \in t \bullet P(x)$

Algoritm inducție structurală

Pasul 0: Se alege variabila după care se face inducție.

Pasul 1 (cazul de bază): Corespunde cazului de bază din schema inducției matematice. În cazul inductiei structurale, se arată că proprietatea P este respectată de **toți constructorii nulari și externi**. Acest lucru este necesar deoarece constructorii nulari și externi sunt de dimensiune 1 (nu primesc ca argument tipul **t** deci nu incrementează dimensiunea).

Pasul 2 (ipoteza inductivă): Se presupune că, **pentru fiecare constructor intern**, proprietatea P **este respectată de parametrii de tip t ai fiecărui constructor intern**.

Pasul 3 (pas de inductie): Folosind ipoteza inductivă, se arată că proprietatea P se respectă pe fiecare **constructor intern** (aceștia fiind de dimensiune mai mare ca 1).

Observații generale

- tabela de advăr pentru operatorul → (implicație) arată în felul următor:

$x1$	$x2$	$x1 \rightarrow x2$
False	False	True
False	True	True
True	False	False
True	True	True

- așadar când aveți de demonstrat o implicație discutați cazul când premisa (x1) este falsă, menționând că în acest caz valoarea de adevăr a concluziei (x2) nu mai contează, apoi arătați că atunci când premisa (x1) este adevărată va rezulta automat că concluzia (x2) este adevărată.
- precizați clar axiomele folosite
- demonstrația se face pentru **fiecare constructor în parte** (cazul de bază pentru fiecare constructor nular și extern, pasul de inducție pentru fiecare constructor intern, cu propria ipoteză inductivă)
- inducția se face după **o singură variabilă**
- când nu mai puteți avansa cu demonstrația, aveți de **demonstrat o nouă proprietate prin inducție structurală**. Alegeți cea mai generală proprietate care vă ajută să avansați și **demonstrați-o tot prin inducție structurală, asemenea cerinței**
- trebuie să folosiți ipoteza inductivă la un moment dat
- dacă aveți mai multe valori de adevăr, împărțiți problema în cazuri

Exerciții rezolvate

Exercițiul 1

Fie TDA-ul List descris de fișa următoare:

Nume: List

Tipuri importate: Nat, Elem

Constructori de bază:

- $[] :\rightarrow List$ # orice listă este fie vidă
- $: : Elem * List \rightarrow List$ # fie un element adăugat la o altă listă
  # sintaxa (x:xs) = elem x adăugat la lista xs
  # constructorul este simbolizat prin simbolul ':', celălalt simbol ':' semnifică că urmează definirea operatorului

Operatori:

- $length : List \rightarrow Nat$ # lungimea unei liste
- $++ : List * List \rightarrow List$ # $A++B$ = concatenarea listelor A și B

Axiome:

- $length([]) = 0$ (LEN1)
- $length(x : xs) = 1 + length(xs)$ (LEN2) # shortcut pentru succ(length(xs))
- $[] ++ L = L$ (APP1)
- $(x : xs) ++ L = x : (xs ++ L)$ (APP2)

Cerință: Să se demonstreze, prin inducție structurală, că operația ++ **e asociativă**.
Trebuie demonstrat, așadar, că (A++B)++C = A++(B++C), oricare ar fi A, B, C TDA-uri de tipul List. Vom face inducție structurală dupa A.

Caz de bază:
$A = []$ (constructor nular)
Trebuie arătat că:
$([] ++ B) ++ C = [] ++ (B ++ C) \Leftarrow (APP1) \Rightarrow$
$B ++ C = B ++ C$
(A) oricare ar fi B și C Liste

Pas de inducție:
$A = (x : xs)$ (constructor intern)

Ipoteza inductivă:
$(xs ++ B) ++ C = xs ++ (B ++ C)$ (II)
Trebuie arătat că:
$((x : xs) ++ B) ++ C = (x : xs) ++ (B ++ C) \Leftarrow (APP2) \Rightarrow$
$(x : (xs ++ B)) ++ C = x : (xs ++ (B ++ C)) \Leftarrow (APP2) \Rightarrow$
$x : ((xs ++ B) ++ C) = x : (xs ++ (B ++ C)) \Leftarrow (II) \Rightarrow$
$x : (xs ++ (B ++ C)) = x : (xs ++ (B ++ C))$
(A)
Întrucât am epuizat toți constructorii de bază, rezultă că ++ e asociativ.

Exercitiul 2

Fie TDA-ul SET descris de:

Nume: SET

Tipuri importate: Bool, T

Constructori de bază:

- $[] :\rightarrow SET$ #
- $: : T \times SET \rightarrow SET$ # $a : x$ reprezintă mulțimea formată prin adăugarea elementului a la mulțimea x

Operatori:

- $maj : T \times SET \rightarrow Bool$ # majorant
- $union : SET \times SET \rightarrow SET$ # reuniunea a 2 mulțimi

Axiome:

- $maj(e, []) = true$ (J1)
- $maj(e, a : x) = (e >= a) \wedge maj(e, x)$ (J2)
- $union([], m) = m$ (U1)
- $union(a : x, m) =$ if $member(a, m)$ then $union(x, m)$ else $a : union(x, m)$ (U2)
- $member(e, []) = false$ (M1)
- $member(e, a : x) = (e == a) \vee member(e, x)$ (M2)

Cerință: Să se demonstreze, prin inducție structurală:
$maj(e, union(m1, m2)) \implies maj(e, m1) \wedge maj(e, m2)$.

Vom face inducție structurală după $m1$.

Caz de bază:
$m1 = []$ (constructor nular)
Trebuie aratat că:
$maj(e, union([], m2)) \implies maj(e, []) \wedge maj(e, m2) \Leftarrow (U1, J1) \Rightarrow$
$maj(e, m2) \Rightarrow true \wedge maj(e, m2)$

Pas de inducție:
$m1 = (a : x)$ (constructor intern)

Ipoteza inductivă:
$maj(e, union(x, m2)) \Rightarrow maj(e, x) \wedge maj(e, m2)$ (II)
Demonstram ca:
$maj(e, union(a : x, m2)) \Rightarrow maj(e, a : x) \wedge maj(e, m2)$

1. Pentru **member(a, m2) = false** $\Leftarrow (U2) \Rightarrow$
   $maj(e, a : union(x, m2)) \Rightarrow maj(e, a : x) \wedge maj(e, m2)$

   (a) Pentru $e \geq a \Leftarrow (J2) \Rightarrow$
   $maj(e, union(x, m2)) \Rightarrow maj(e, x) \wedge maj(e, m2) \Leftarrow (II) \Rightarrow$
   $True$.

   (b) Pentru $e < a \Leftarrow (J2) \Rightarrow$
   $False \Rightarrow ceva$.

2. Pentru **member(a, m2) = True** $\Leftarrow (U2, J2) \Rightarrow$
   $maj(e, union(x, m2)) \Rightarrow e \geq a \wedge maj(e, x) \wedge maj(e, m2) \Leftarrow (II, P1 - \text{demonstrata ulterior}) \Rightarrow True.$

In continuare demonstram P1: $member(a, m2) \wedge maj(e, m2) \Rightarrow e \geq a$**.**

Vom face inducție structurala după $m2$.

Caz de bază:
$m2 = []$ (constructor nular)
Trebuie aratat că:
$member(a, []) \wedge maj(e, []) \Rightarrow e \geq a \Leftarrow (M1) \Rightarrow$
$False \Rightarrow ceva$

Pas de inducție:
$m2 = (b : y)$ (constructor intern)

Ipoteza inductivă:
$member(a, y) \wedge maj(e, y) \Rightarrow e \geq a \ (II)$
Demonstram ca:
$member(a, b : y) \wedge maj(e, b : y) \Rightarrow e \geq a \Leftarrow (M2, J2) \Rightarrow$
$(a == b \vee member(a, y)) \wedge (e \geq b) \wedge maj(e, y) \Rightarrow e \geq a$

1. Pentru **a==b**
   $true \wedge e \geq a \wedge maj(e, y) \Rightarrow e \geq a.$

2. Pentru **member(a, y) = True**
   $member(a, y) \wedge e \geq b \wedge maj(e, y) \Rightarrow e \geq a \Leftarrow (II) \Rightarrow True$

Exerciții propuse

1. O expresie aritmetică complet parantezată este definită astfel:
   $0, 1, x, [e1 + e2], [e1 * e2], [-e2]$
   3 constructori nulari (primii 3 enumerați mai sus), 3 constructori interni (ultimii 3 enumerați mai sus).
   Să notăm cu E acest tip de date.
   Se definesc operatorii:
   $\boldsymbol{eval(e, n) : E * N \to N}$

   - (E1) $eval(0, n) = 0$
   - (E2) $eval(1, n) = 1$
   - (E3) $eval(x, n) = n$
   - (E4) $eval([e1 + e2], n) = eval(e1, n) + eval(e2, n)$
   - (E5) $eval([e1 * e2], n) = eval(e1, n) * eval(e2, n)$
   - (E6) $eval([-e1], n) = -eval(e1, n)$

   $\boldsymbol{subst(e, f) : E * E \to E}$

   - (S1) $subst(0, f) = 0$
   - (S2) $subst(1, f) = 1$
   - (S3) $subst(x, f) = f$
   - (S4) $subst([e1 + e2], f) = [subst(e1, f) + subst(e2, f)]$
   - (S5) $subst([e1 * e2], f) = [subst(e1, f) * subst(e2, f)]$
   - (S6) $subst([-e], f) = [-subst(e, f)]$

   Operatorul eval(e, n) substituie în expresia e fiecare apariție a lui x cu n și întoarce rezulatul ei.
   Operatorul subst(e, f) substituie în expresie e fiecare apariție a lui x cu f și întoarce noua expresie formată.
   Să se demonstreze prin inductie structurala ca, pentru orice e, f $\in$ E și n $\in \mathbb{N}$, proprietatea urmatoare este adevarata:
   eval(subst(e, f), n)= eval(e, eval(f, n))

2. Fie tipul de date T LIST, definit prin constructorii de bază:

   - $[] :\to TLIST$
   - $: : T * TLIST \to TLIST$

   Se consideră **operatorii**:
   $\boldsymbol{head(l) : TLIST \backslash [] \to T}$

   - head(a:l) = a

   $\boldsymbol{range(l) : TLIST \to BOOL}$

   - $range([]) = false$
   - $range(a : []) = (a == 1)$
   - $range(a : l) = (a - 1 == head(l))\&\&range(l)$, pentru $l != []$

   $\boldsymbol{sum(l) : TLIST \to \mathbb{N}}$

   - $sum([]) = 0$
   - $sum(a : l) = a + sum(l)$

   Demonstraţi următoarea proprietate prin inducţie structurală:
   $\boldsymbol{range(l) \to sum(l) == head(l) * (head(l) + 1)/2}$

3. Fie tipul de date LIST care reprezintă mulțimea listelor cu numere întregi definit prin următorii constructori de bază:

- $[] :\rightarrow LIST$
- $(x : l) : INT * LIST \rightarrow LIST$

Fie FUN mulțimea funcțiilor definite astfel: $INT * INT \rightarrow INT$.
Se definesc funcțiile (operatori) următoare, cu axiomele aferente:

$\boldsymbol{fold : FUN * INT * LIST \rightarrow INT}$

- $fold(f, z, []) = z$
- $fold(f, z, x : l) = f(x, fold(f, z, l))$

$\boldsymbol{len : LIST \rightarrow INT}$

- $len([]) = 0$
- $len(x : l) = 1 + len(l)$

Să se demonstreze prin inducție structurală că P(l) este adevarată, pentru $\forall$ l $\in$ LIST:
$\boldsymbol{P(l) = (len(l) = fold(inc, 0, l))}$
cu inc definită astfel $inc(x, y) = y + 1$

4. Fie tipul de date asbtract BinTree definit prin constructorii de bază:

   $empty : \rightarrow BinTree$ # Arborele binar vid

   $node : BinTree * Elem * BinTree \rightarrow BinTree$ # Subarborele stâng, radăcina, sub drept

   Se consideră operatorii:

   $\boldsymbol{size : BinTree \rightarrow N}$ # Numărul de elemente din arbore

   $\boldsymbol{all : (Elem \rightarrow Bool) * BinTree \rightarrow Bool}$

   # Întoarce True dacă p(x) este adevărat pentru toate elementele x din Tree, False altfel

   Axiomele pentru All:
   (All1): $all(p, empty) = True$
   (All2): $all(p, node(left, root, right)) = p(root)$ && $all(p, left)$ && $all(p, right)$

   $\boldsymbol{countif : (Elem \rightarrow Bool) * BinTree \rightarrow N}$

   # countif(p, tree) = câte elemente din tree satisfac predicatul p (adică pentru câte elemente din tree p(x) este adevărat

   Scrieți axiomele pentru operatorii size și countif, apoi demonstrați prin inducție structurală:

$$\boldsymbol{all(p, t) \rightarrow countif(p, t) == size(t)} \quad (1)$$